Ticketium — Error Handling Scenarios

Team x53AIBoosted · Version 0.1 · 2026-10-23

This document walks through every error response of the 11 Ticketium API endpoints. For each endpoint and each error it gives the **situation** that causes the error, an **example request**, the **error response**, and the expected **client reaction** in the Web SPA.

It complements the Error Handling design, which defines the rules and the catalogue of problem types, and the OpenAPI specification, which is the machine-readable contract.

Conventions used below:

- Base URL: `https://api.ticketium.example/api/v1`.
- Every error body is an RFC 9457 Problem Details object sent as `application/problem+json`. Clients branch on `type`; `detail` is shown to the user but never parsed.
- `traceId` values are shortened (…) for readability.
- Example IDs: event `3fa85f64-5717-4562-b3fc-2c963f66afa6`, hold `9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b`, venue `7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f`.

1. Endpoint Overview

#	Endpoint	Success	Error codes
1	`POST /auth/sign-up`	201	400, 409, 429, 500
2	`POST /auth/sign-in`	200	400, 401, 429, 500
3	`GET /events`	200	400, 429, 500
4	`POST /events`	201	400, 401, 403, 404, 422, 429, 500
5	`GET /events/{eventId}`	200	400, 404, 429, 500
6	`PUT /events/{eventId}`	200	400, 401, 403, 404, 409, 412, 422, 428, 429, 500
7	`POST /events/{eventId}/cancellation`	201	400, 401, 403, 404, 409, 429, 500
8	`POST /events/{eventId}/holds`	201	400, 401, 404, 409, 422, 429, 500, 503
9	`DELETE /holds/{holdId}`	204	401, 404, 409, 429, 500
10	`POST /holds/{holdId}/payment`	201	400, 401, 402, 404, 409, 410, 429, 500, 502, 503, 504
11	`GET /me/tickets`	200	400, 401, 429, 500

2. Errors Common to All Endpoints

`429 Too Many Requests` and `500 Internal Server Error` can be returned by any endpoint. They behave the same everywhere, so they are described once here; the endpoint sections

show them only where the situation is specific (e.g. brute-force protection on sign-in).

2.1 Any endpoint — 429 Too Many Requests

Situation: A client exceeds the rate limit, e.g. a bot refreshes an event page many times per second during a high-demand ticket drop (SRS NFR-5).

Request:

```
GET /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
```

Response:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 5

{
  "type": "https://ticketium.example/problems/rate-limited",
  "title": "Too many requests",
  "status": 429,
  "detail": "Rate limit of 10 requests per second exceeded.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-a1b2c3d4…-01"
}
```

Client reaction: Do not retry immediately. Disable the action that triggered the request, show "Too many requests, please wait N seconds" with a countdown taken from `Retry-After`, then retry once with exponential backoff and jitter.

2.2 Any endpoint — 500 Internal Server Error

Situation: An unexpected exception occurs on the server, e.g. a bug in the search query builder. No internal details (stack trace, SQL) are returned.

Request:

```
GET /api/v1/events?q=jazz HTTP/1.1
```

Response:

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/internal-error",
  "title": "An unexpected error occurred",
  "status": 500,
  "detail": "Something went wrong on our side. Please try again later and quote
the traceId if the problem persists.",
  "instance": "/api/v1/events",
  "traceId": "00-b2c3d4e5…-01"
}
```

Client reaction: Show a generic error message together with the `traceId` so the user can report it. Offer a "Try again" button; do not retry automatically more than once.

3. `POST /auth/sign-up`

3.1 400 Bad Request

Situation: The sign-up form is submitted with an invalid email and a password shorter than 8 characters.

Request:

```
POST /api/v1/auth/sign-up HTTP/1.1
Content-Type: application/json

{ "email": "maksym@", "password": "1234", "role": "Buyer" }
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 2 invalid values.",
  "instance": "/api/v1/auth/sign-up",
  "traceId": "00-5c6d7e8f…-01",
  "errors": [
    { "pointer": "#/email", "detail": "Must be a valid email address." },
    { "pointer": "#/password", "detail": "Must be at least 8 characters long." }
  ]
}
```

Client reaction: Map each `errors[].pointer` to its form field, highlight the fields and show the messages under them. Keep the entered values. Do not resend until the user changes the data.

3.2 409 Conflict

Situation: A user tries to sign up with an email that already has an account.

Request:

```
POST /api/v1/auth/sign-up HTTP/1.1
Content-Type: application/json

{ "email": "maksym@example.com", "password": "S3cure-pass", "role": "Buyer" }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/email-already-registered",
  "title": "Email is already registered",
  "status": 409,
  "detail": "An account with email 'maksym@example.com' already exists.",
  "instance": "/api/v1/auth/sign-up",
  "traceId": "00-4bf92f35…-01"
}
```

Client reaction: Show the message under the email field with links "Sign in instead" and "Forgot password?". Pre-fill the email on the sign-in page if the user follows the link.

4. `POST /auth/sign-in`

4.1 400 Bad Request

Situation: The request body is missing the `password` field (a front-end bug or a hand-crafted request).

Request:

```
POST /api/v1/auth/sign-in HTTP/1.1
Content-Type: application/json

{ "email": "maksym@example.com" }
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 1 invalid value.",
  "instance": "/api/v1/auth/sign-in",
  "traceId": "00-6e7f8091…-01",
  "errors": [
    { "pointer": "#/password", "detail": "The password field is required." }
  ]
}
```

Client reaction: Stop the loading state of the "Sign in" button and highlight the empty field with "Fill in all required fields". Do not resend until the field is filled.

4.2 401 Unauthorized

Situation: The password is wrong, or no account with this email exists. Both cases return the same response so that attackers cannot find out which emails are registered.

Request:

```
POST /api/v1/auth/sign-in HTTP/1.1
Content-Type: application/json

{ "email": "maksym@example.com", "password": "wrong-password" }
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/invalid-credentials",
  "title": "Invalid credentials",
  "status": 401,
  "detail": "The email or password is incorrect.",
  "instance": "/api/v1/auth/sign-in",
  "traceId": "00-1c2b3a4d…-01"
}
```

Client reaction: Show "The email or password is incorrect" above the form, clear only the password field and keep the email. Do not say which of the two was wrong.

4.3 429 Too Many Requests

Situation: Somebody tries to guess a password and sends 10 failed sign-in attempts for the same email within a minute.

Request:

```
POST /api/v1/auth/sign-in HTTP/1.1
Content-Type: application/json

{ "email": "maksym@example.com", "password": "guess-11" }
```

Response:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 300

{
  "type": "https://ticketium.example/problems/rate-limited",
  "title": "Too many requests",
  "status": 429,
  "detail": "Too many failed sign-in attempts. Try again in 5 minutes.",
  "instance": "/api/v1/auth/sign-in",
  "traceId": "00-2d3e4f50…-01"
}
```

Client reaction: Disable the "Sign in" button and show a countdown based on `Retry-After`. Offer the "Forgot password?" link.

5. `GET /events`

5.1 400 Bad Request

Situation: The user picks a date range where "to" is earlier than "from", and the page size in the URL was edited to an out-of-range value.

Request:

```
GET /api/v1/events?from=2026-12-31T00:00:00Z&to=2026-12-01T00:00:00Z&pageSize=500
HTTP/1.1
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request query contains 2 invalid parameters.",
  "instance": "/api/v1/events",
  "traceId": "00-8a7b6c5d…-01",
  "errors": [
    { "parameter": "to", "detail": "Must be later than 'from'." },
    { "parameter": "pageSize", "detail": "Must be between 1 and 100." }
  ]
}
```

Client reaction: Highlight the date filter and show the message next to it. Reset `pageSize` to the default value and keep the other filters.

429 and 500 — see §2.

6. `POST /events`

6.1 400 Bad Request

Situation: An organizer submits the "New event" form with an empty title and a malformed currency code.

Request:

```
POST /api/v1/events HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{
  "venueId": "7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f",
  "title": "",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "uah" }
}
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 2 invalid values.",
  "instance": "/api/v1/events",
  "traceId": "00-3b4c5d6e…-01",
  "errors": [
    { "pointer": "#/title", "detail": "Must be between 3 and 200 characters long."
},
    { "pointer": "#/price/currency", "detail": "Must be a 3-letter ISO 4217 code
in upper case." }
  ]
}
```

Client reaction: Highlight the title and currency fields with the messages under them. Keep all other entered data.

6.2 401 Unauthorized

Situation: The organizer filled the form for a long time and the access token expired.

Request:

```
POST /api/v1/events HTTP/1.1
Authorization: Bearer <expired_token>
Content-Type: application/json

{
  "venueId": "7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f",
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" }
}
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="invalid_token", error_description="The access
token expired"

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "The access token expired at 2026-10-23T13:00:00Z.",
  "instance": "/api/v1/events",
  "traceId": "00-4c5d6e7f…-01"
}
```

Client reaction: Save the form data as a local draft, remove the expired token, redirect to sign-in with "Your session has expired". After signing in, restore the draft and let the organizer submit again.

6.3 403 Forbidden

Situation: A user with the `Buyer` role calls the endpoint (e.g. by crafting the request manually); only organizers can create events.

Request:

```
POST /api/v1/events HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{
  "venueId": "7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f",
  "title": "My Party",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 100.00, "currency": "UAH" }
}
```

Response:

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/forbidden",
  "title": "Access denied",
  "status": 403,
  "detail": "Only users with the 'Organizer' role can create events.",
  "instance": "/api/v1/events",
  "traceId": "00-2f3e4d5c…-01"
}
```

Client reaction: Show "You don't have permission to create events". The SPA normally hides the "New event" button for buyers, so this case also indicates a UI bug and is logged.

6.4 404 Not Found

Situation: The selected venue was deleted by an administrator while the organizer was filling in the form.

Request:

```
POST /api/v1/events HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{
  "venueId": "7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f",
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" }
}
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Venue '7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f' was not found.",
  "instance": "/api/v1/events",
  "traceId": "00-3a4b5c6d…-01",
  "resource": "venue"
}
```

Client reaction: Because `resource` is `venue`, reload the venue list, clear the venue selector and ask the organizer to choose another venue. Keep the other fields.

6.5 422 Unprocessable Content

Situation: The request is valid, but the venue already hosts another event at the same time.

Request:

```
POST /api/v1/events HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{
  "venueId": "7d1c0a52-3c8e-4d6b-9f7e-1a2b3c4d5e6f",
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" }
}
```

Response:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/business-rule-violation",
  "title": "Business rule violated",
  "status": 422,
  "detail": "Venue 'Main Hall' is already booked for another event at 2026-12-
31T19:00:00Z.",
  "instance": "/api/v1/events",
  "traceId": "00-4b5c6d7e…-01",
  "rule": "venue-double-booking"
}
```

Client reaction: Use `rule` to highlight the venue and start-time fields together and show the `detail`. Suggest picking another time or venue.

7. `GET /events/{eventId}`

7.1 400 Bad Request

Situation: A shared link was truncated, so the event id in the URL is not a valid UUID.

Request:

```
GET /api/v1/events/3fa85f64-5717 HTTP/1.1
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 1 invalid value.",
  "instance": "/api/v1/events/3fa85f64-5717",
  "traceId": "00-5d6e7f80…-01",
  "errors": [
    { "parameter": "eventId", "detail": "Must be a valid UUID." }
  ]
}
```

Client reaction: Show a "Broken link" page with a button to the event catalogue.

7.2 404 Not Found

Situation: The user opens an old link to an event that does not exist (wrong id or removed test data).

Request:

```
GET /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Event '3fa85f64-5717-4562-b3fc-2c963f66afa6' was not found.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-7e8f90a1…-01",
  "resource": "event"
}
```

Client reaction: Show "This event is no longer available" and redirect to the catalogue. Cancelled events are **not** 404 — they are returned with status `Cancelled`.

429 and 500 — see §2.

8. `PUT /events/{eventId}`

8.1 400 Bad Request

Situation: The organizer sets the sales close time after the event start time.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <organizer_token>
If-Match: "7"
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "salesCloseAt": "2026-12-31T21:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 1 invalid value.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-6f708192…-01",
  "errors": [
    { "pointer": "#/salesCloseAt", "detail": "Must not be later than 'startsAt'."
}
  ]
}
```

Client reaction: Highlight the "Sales close" field with the message and keep all edits.

8.2 401 Unauthorized

Situation: The organizer's token expired while editing the event.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <expired_token>
If-Match: "7"
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 800.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="invalid_token", error_description="The access
token expired"

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "The access token expired at 2026-10-23T13:00:00Z.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-708192a3…-01"
}
```

Client reaction: Save the edits as a local draft, redirect to sign-in, and restore the draft afterwards.

8.3 403 Forbidden

Situation: An organizer tries to edit an event that belongs to another organizer.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <other_organizer_token>
If-Match: "7"
Content-Type: application/json

{
  "title": "Renamed Event",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/forbidden",
  "title": "Access denied",
  "status": 403,
  "detail": "Only the organizer of this event can modify it.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-5c6d7e8f…-01"
}
```

Client reaction: Show "You can only edit your own events" and return to the organizer's event list.

8.4 404 Not Found

Situation: The organizer edits an event using an id that does not exist (e.g. a stale browser tab after test data was reset).

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <organizer_token>
If-Match: "7"
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Event '3fa85f64-5717-4562-b3fc-2c963f66afa6' was not found.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-8192a3b4…-01",
  "resource": "event"
}
```

Client reaction: Close the edit form, remove the event from the organizer's list and show "This event no longer exists".

8.5 409 Conflict

Situation: The organizer tries to edit an event that has already been cancelled.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <organizer_token>
If-Match: "9"
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2027-01-15T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/event-not-modifiable",
  "title": "Event cannot be modified",
  "status": 409,
  "detail": "The event was cancelled on 2026-10-20 and can no longer be edited.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-6d7e8f90…-01"
}
```

Client reaction: Reload the event, switch the page to read-only mode and show the "Cancelled" badge. Suggest creating a new event instead.

8.6 412 Precondition Failed

Situation: Two people from the organizer's team edit the same event. The first one saves successfully and the event version becomes `"8"`; the second one still sends `If-Match: "7"`.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <organizer_token>
If-Match: "7"
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 700.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 412 Precondition Failed
Content-Type: application/problem+json
ETag: "8"

{
  "type": "https://ticketium.example/problems/precondition-failed",
  "title": "Resource was modified",
  "status": 412,
  "detail": "The event was changed by someone else. Reload it and apply your
changes again.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-8f90a1b2…-01"
}
```

Client reaction: Fetch the latest version, show what changed next to the user's own edits and let the user re-apply them. Never overwrite silently.

8.7 422 Unprocessable Content

Situation: The organizer reduces the capacity below the number of tickets already sold.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <organizer_token>
If-Match: "8"
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" },
  "capacity": 100
}
```

Response:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/business-rule-violation",
  "title": "Business rule violated",
  "status": 422,
  "detail": "Capacity cannot be reduced to 100 because 142 tickets are already
sold.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-7e8f90a1…-01",
  "rule": "capacity-below-sold"
}
```

Client reaction: Highlight the capacity field, show the `detail` and set the field's minimum to the number of sold tickets.

8.8 428 Precondition Required

Situation: A client (e.g. a script) sends the update without the `If-Match` header.

Request:

```
PUT /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6 HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{
  "title": "New Year Jazz Night",
  "startsAt": "2026-12-31T19:00:00Z",
  "price": { "amount": 750.00, "currency": "UAH" },
  "capacity": 300
}
```

Response:

```
HTTP/1.1 428 Precondition Required
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/precondition-required",
  "title": "Precondition required",
  "status": 428,
  "detail": "Send the event's ETag in the If-Match header.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "traceId": "00-90a1b2c3…-01"
}
```

Client reaction: This is a client bug. The SPA reads the `ETag` from `GET /events/{eventId}` and always sends it; the error is logged and the user sees a generic error.

9. `POST /events/{eventId}/cancellation`

9.1 400 Bad Request

Situation: The organizer confirms the cancellation without giving a reason.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{ "reason": "" }
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 1 invalid value.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation",
  "traceId": "00-a1b2c3d4…-01",
  "errors": [
    { "pointer": "#/reason", "detail": "Must be between 3 and 500 characters
long." }
  ]
}
```

Client reaction: Keep the confirmation dialog open and highlight the "Reason" field — buyers will see this text in the cancellation notice.

9.2 401 Unauthorized

Situation: The organizer's session expired before confirming the cancellation.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation HTTP/1.1
Authorization: Bearer <expired_token>
Content-Type: application/json

{ "reason": "The headliner is ill; the concert will not take place." }
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="invalid_token", error_description="The access
token expired"

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "The access token expired at 2026-10-23T13:00:00Z.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation",
  "traceId": "00-b2c3d4e5…-01"
}
```

Client reaction: Save the entered reason, redirect to sign-in and reopen the confirmation dialog afterwards. Do **not** cancel automatically after sign-in — the organizer must confirm again.

9.3 403 Forbidden

Situation: An organizer tries to cancel another organizer's event.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation HTTP/1.1
Authorization: Bearer <other_organizer_token>
Content-Type: application/json

{ "reason": "Not needed anymore." }
```

Response:

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/forbidden",
  "title": "Access denied",
  "status": 403,
  "detail": "Only the organizer of this event can cancel it.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation",
  "traceId": "00-5c6d7e8f…-01"
}
```

Client reaction: Close the dialog and show "You can only cancel your own events".

9.4 404 Not Found

Situation: The event id does not exist.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{ "reason": "The headliner is ill; the concert will not take place." }
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Event '3fa85f64-5717-4562-b3fc-2c963f66afa6' was not found.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation",
  "traceId": "00-c3d4e5f6…-01",
  "resource": "event"
}
```

Client reaction: Close the dialog, remove the event from the list and show "This event no longer exists".

9.5 409 Conflict — event already cancelled

Situation: The organizer double-clicks "Cancel event", or a co-organizer already cancelled it from another device.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{ "reason": "The headliner is ill; the concert will not take place." }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/event-already-cancelled",
  "title": "Event is already cancelled",
  "status": 409,
  "detail": "The event was already cancelled on 2026-10-20T09:15:00Z.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation",
  "traceId": "00-6d7e8f90…-01"
}
```

Client reaction: Treat it as an informational message, not a failure: reload the event, show the "Cancelled" badge and the existing cancellation details.

9.6 409 Conflict — event already started

Situation: The organizer tries to cancel an event after its start time.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation HTTP/1.1
Authorization: Bearer <organizer_token>
Content-Type: application/json

{ "reason": "Technical problems at the venue." }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/event-already-started",
  "title": "Event has already started",
  "status": 409,
  "detail": "The event started at 2026-12-31T19:00:00Z and can no longer be
cancelled.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/cancellation",
  "traceId": "00-7e8f90a1…-01"
}
```

Client reaction: Show the `detail` and hide the "Cancel event" button for this event. Refunds after the start are handled by support.

10. `POST /events/{eventId}/holds`

10.1 400 Bad Request

Situation: The client sends more seats than allowed in one request and repeats a seat label.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "seats": ["A-1", "A-2", "A-3", "A-4", "A-5", "A-6", "A-6"] }
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 1 invalid value.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-d4e5f607…-01",
  "errors": [
    { "pointer": "#/seats", "detail": "Must contain from 1 to 6 unique seat
labels." }
  ]
}
```

Client reaction: Show "You can select up to 6 different seats". The seat map should already prevent this, so the case is also logged as a UI bug.

10.2 401 Unauthorized

Situation: An anonymous visitor selects seats and clicks "Checkout" without signing in.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Content-Type: application/json

{ "seats": ["A-12", "A-13"] }
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "An access token is required to hold seats.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-e5f60718…-01"
}
```

Client reaction: Remember the selected seats, redirect to sign-in, then return to the seat map and retry the hold. If some seats were taken meanwhile, the retry returns `409` (see 10.4).

10.3 404 Not Found

Situation: The event id in the request does not exist.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "seats": ["A-12", "A-13"] }
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Event '3fa85f64-5717-4562-b3fc-2c963f66afa6' was not found.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-f6071829…-01",
  "resource": "event"
}
```

Client reaction: Show "This event is no longer available" and redirect to the catalogue.

10.4 409 Conflict — seat unavailable

Situation: Two buyers select seat A-12 at the same moment. The first buyer acquires the seat lock in the seat-lock store; the second buyer's lock on A-12 is denied, so any locks already

taken for the second buyer's other seats are released and none of them are held (all-or-nothing).

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "seats": ["A-12", "A-13"] }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/seat-unavailable",
  "title": "Seat is not available",
  "status": 409,
  "detail": "1 of 2 requested seats is already held or sold.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-8f90a1b2…-01",
  "unavailableSeats": ["A-12"]
}
```

Client reaction: Mark the seats from `unavailableSeats` as taken on the seat map, keep the other selected seats (A-13) selected, show "Seat A-12 was just taken — please choose another one" and refresh availability.

10.5 409 Conflict — event not on sale

Situation: The buyer had the seat map open, but ticket sales closed (or the event was cancelled or sold out) before they clicked "Checkout".

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "seats": ["A-12", "A-13"] }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/event-not-on-sale",
  "title": "Event is not on sale",
  "status": 409,
  "detail": "Ticket sales for this event closed at 2026-12-31T18:00:00Z.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-90a1b2c3…-01"
}
```

Client reaction: Reload the event, disable the seat map and show the `detail` with the event's new status ("Sales closed", "Sold out" or "Cancelled").

10.6 422 Unprocessable Content

Situation: The buyer already holds 4 seats for this event in another browser tab and now tries to hold 3 more; the limit is 6 seats per buyer per event.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "seats": ["B-1", "B-2", "B-3"] }
```

Response:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/hold-limit-exceeded",
  "title": "Hold limit exceeded",
  "status": 422,
  "detail": "You already hold 4 seats for this event; at most 6 seats per buyer
are allowed.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-a1b2c3d4…-01",
  "limit": 6,
  "current": 4
}
```

Client reaction: Show "You can select at most 2 more seats" (computed as `limit - current`) and offer a link to the existing checkout.

10.7 429 Too Many Requests

Situation: A scalping bot sends dozens of hold requests per second at a ticket drop.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <bot_token>
Content-Type: application/json

{ "seats": ["A-1"] }
```

Response:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 10

{
  "type": "https://ticketium.example/problems/rate-limited",
  "title": "Too many requests",
  "status": 429,
  "detail": "Rate limit of 10 requests per second exceeded.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-a1b2c3d4…-02"
}
```

Client reaction: For a real user, disable the "Checkout" button for `Retry-After` seconds and show a countdown. Requests are not retried automatically.

10.8 503 Service Unavailable

Situation: The seat-lock store is unreachable during a ticket drop, so seat locks cannot be acquired. The API fails closed: it never falls back to booking without a lock.

Request:

```
POST /api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "seats": ["A-12", "A-13"] }
```

Response:

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/problem+json
Retry-After: 3

{
  "type": "https://ticketium.example/problems/service-unavailable",
  "title": "Service temporarily unavailable",
  "status": 503,
  "detail": "Seat reservations are temporarily unavailable. Please retry
shortly.",
  "instance": "/api/v1/events/3fa85f64-5717-4562-b3fc-2c963f66afa6/holds",
  "traceId": "00-c3d4e5f6…-01"
}
```

Client reaction: Keep the seat selection, show "High demand — retrying…" and retry after `Retry-After` seconds with backoff (at most 3 attempts), then ask the user to try again later.

500 — see §2.

11. `DELETE /holds/{holdId}`

11.1 401 Unauthorized

Situation: The buyer clicks "Cancel checkout" after the session expired.

Request:

```
DELETE /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b HTTP/1.1
Authorization: Bearer <expired_token>
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="invalid_token", error_description="The access
token expired"

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "The access token expired at 2026-10-23T13:00:00Z.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b",
  "traceId": "00-07182930…-01"
}
```

Client reaction: Leave the checkout page and redirect to sign-in. No retry is needed: the hold will expire by itself at `expiresAt`.

11.2 404 Not Found

Situation: The hold id is unknown, or it belongs to another user. Both cases return `404` so that hold ids of other buyers cannot be discovered.

Request:

```
DELETE /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b HTTP/1.1
Authorization: Bearer <buyer_token>
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Hold '9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b' was not found.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b",
  "traceId": "00-18293a4b…-01",
  "resource": "hold"
}
```

Client reaction: Treat it as already released: clear the local checkout state and return to the seat map.

11.3 409 Conflict

Situation: The buyer paid in one browser tab and then clicks "Cancel checkout" in another.

Request:

```
DELETE /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b HTTP/1.1
Authorization: Bearer <buyer_token>
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/hold-already-paid",
  "title": "Hold is already paid",
  "status": 409,
  "detail": "Hold has been paid and converted into tickets; request a refund
instead.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b",
  "traceId": "00-b2c3d4e5…-02"
}
```

Client reaction: Show "These seats are already paid" and redirect to "My tickets".

429 and 500 — see §2.

12. `POST /holds/{holdId}/payment`

All requests to this endpoint carry an `Idempotency-Key` header, so a retry after a network error never charges the buyer twice.

12.1 400 Bad Request

Situation: The client forgets the `Idempotency-Key` header.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request contains 1 invalid value.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-c3d4e5f6…-02",
  "errors": [
    { "header": "Idempotency-Key", "detail": "Header is required and must be a
UUID." }
  ]
}
```

Client reaction: Client bug: log it and show a generic error. The SPA generates a new UUID once per "Pay" click and reuses it for retries of that click.

12.2 401 Unauthorized

Situation: The session expired while the buyer was entering card details.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <expired_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="invalid_token", error_description="The access
token expired"

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "The access token expired at 2026-10-23T13:00:00Z.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-293a4b5c…-01"
}
```

Client reaction: Open sign-in in a modal so the checkout page and the hold countdown stay visible. After sign-in, retry with the **same** `Idempotency-Key` if the hold has not expired.

12.3 402 Payment Required

Situation: The card issuer declines the payment because of insufficient funds. Following the saga, the hold is released and the seats go back on sale.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 402 Payment Required
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/payment-declined",
  "title": "Payment was declined",
  "status": 402,
  "detail": "The card issuer declined the payment (insufficient funds).",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-d4e5f607…-02",
  "declineCode": "insufficient_funds",
  "holdReleased": true
}
```

Client reaction: Show a message based on `declineCode` ("Insufficient funds — try another card"). Because `holdReleased` is `true`, stop the countdown and return to the seat map with the previous selection pre-selected so the buyer can hold the seats again.

12.4 404 Not Found

Situation: The hold id is unknown or belongs to another user.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Hold '9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b' was not found.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-3a4b5c6d…-02",
  "resource": "hold"
}
```

Client reaction: Clear the checkout state and return to the seat map with "Your reservation was not found — please select seats again".

12.5 409 Conflict — hold already paid

Situation: The buyer paid on a phone and then presses "Pay" again on a laptop (a new click, so a new `Idempotency-Key`).

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 7a1d9c3f-2b4e-4f6a-8c0d-9e8f7a6b5c4d
Content-Type: application/json

{ "paymentToken": "tok_9KlMn3Cd" }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/hold-already-paid",
  "title": "Hold is already paid",
  "status": 409,
  "detail": "This hold has already been paid; see your tickets.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-e5f60718…-02"
}
```

Client reaction: Show a success-style message "Already paid" and redirect to "My tickets". No second charge happened.

12.6 409 Conflict — idempotency key reused

Situation: Because of a client bug, the same `Idempotency-Key` is sent with a different payment token.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_DIFFERENT" }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/idempotency-key-reused",
  "title": "Idempotency key reused with a different request",
  "status": 409,
  "detail": "Key '5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f' was already used with a
different request body.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-f6071829…-02"
}
```

Client reaction: Log the bug. Generate a new `Idempotency-Key` and ask the user to press "Pay" again.

12.7 409 Conflict — request in progress

Situation: The buyer double-clicks "Pay". The second request with the same `Idempotency-Key` arrives while the first one is still waiting for the payment gateway.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/request-in-progress",
  "title": "Request is already in progress",
  "status": 409,
  "detail": "A payment with this Idempotency-Key is still being processed.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-0718293a…-01"
}
```

Client reaction: Ignore this response and wait for the result of the first request. Keep the "Pay" button disabled with a spinner until then.

12.8 410 Gone

Situation: The buyer took longer than the hold time (e.g. 10 minutes) to enter card details. The hold expired and the seats went back on sale.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 410 Gone
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/hold-expired",
  "title": "Hold has expired",
  "status": 410,
  "detail": "The hold expired at 2026-10-23T14:10:00Z; please select seats
again.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-18293a4b…-02",
  "expiredAt": "2026-10-23T14:10:00Z"
}
```

Client reaction: Show "Your reservation expired" and return to the seat map with the previous seats pre-selected, so the buyer can try to hold them again. The buyer was not charged.

12.9 502 Bad Gateway

Situation: The payment gateway returns a malformed response or an internal error. The charge is treated as failed, any authorization is voided, and the hold is released.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 502 Bad Gateway
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/payment-gateway-error",
  "title": "Payment provider error",
  "status": 502,
  "detail": "The payment provider returned an unexpected response. You were not
charged.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-293a4b5c…-02",
  "holdReleased": true
}
```

Client reaction: Show "Payment provider error, you were not charged" with the `traceId`. Because `holdReleased` is `true`, return to the seat map with the previous selection.

12.10 503 Service Unavailable

Situation: The payment gateway has been failing, so the circuit breaker is open. No charge is attempted and the hold is **kept** until it expires.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/problem+json
Retry-After: 30

{
  "type": "https://ticketium.example/problems/payment-unavailable",
  "title": "Payment is temporarily unavailable",
  "status": 503,
  "detail": "Payments are temporarily unavailable. Your seats stay held until
2026-10-23T14:10:00Z.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-3a4b5c6d…-03",
  "holdReleased": false
}
```

Client reaction: Stay on the checkout page; the countdown keeps running. Show "Payments are temporarily unavailable, retrying in 30 s" and retry automatically after `Retry-After` with the **same** `Idempotency-Key` while the hold is still active.

12.11 504 Gateway Timeout

Situation: The payment gateway does not answer within the timeout, even after retries. The outcome is unknown, so the payment module voids any pending authorization and releases the hold; a reconciliation job refunds any charge that is later found without a ticket.

Request:

```
POST /api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment HTTP/1.1
Authorization: Bearer <buyer_token>
Idempotency-Key: 5f0c7b2e-8d1a-4c3b-9e2f-1a2b3c4d5e6f
Content-Type: application/json

{ "paymentToken": "tok_1QxYz2Ab" }
```

Response:

```
HTTP/1.1 504 Gateway Timeout
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/payment-gateway-timeout",
  "title": "Payment provider timed out",
  "status": 504,
  "detail": "The payment provider did not respond in time. Any pending
authorization will be voided.",
  "instance": "/api/v1/holds/9b2e6f1a-4c3d-4e5f-8a7b-6c5d4e3f2a1b/payment",
  "traceId": "00-4b5c6d7e…-02",
  "holdReleased": true
}
```

Client reaction: Show "The payment could not be confirmed. If your card was charged, the amount will be returned automatically", with the `traceId`. Return to the seat map with the previous selection.

429 and 500 — see §2.

13. `GET /me/tickets`

13.1 400 Bad Request

Situation: The page number in the URL was edited to `0`.

Request:

```
GET /api/v1/me/tickets?page=0 HTTP/1.1
Authorization: Bearer <buyer_token>
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://ticketium.example/problems/validation-error",
  "title": "One or more validation errors occurred",
  "status": 400,
  "detail": "The request query contains 1 invalid parameter.",
  "instance": "/api/v1/me/tickets",
  "traceId": "00-5c6d7e8f…-02",
  "errors": [
    { "parameter": "page", "detail": "Must be greater than or equal to 1." }
  ]
}
```

Client reaction: Silently reset to page 1 and reload the list.

13.2 401 Unauthorized

Situation: The buyer opens "My tickets" from a bookmark after the token expired.

Request:

```
GET /api/v1/me/tickets HTTP/1.1
Authorization: Bearer <expired_token>
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="invalid_token", error_description="The access
token expired"

{
  "type": "https://ticketium.example/problems/unauthenticated",
  "title": "Authentication required",
  "status": 401,
  "detail": "The access token expired at 2026-10-23T13:00:00Z.",
  "instance": "/api/v1/me/tickets",
  "traceId": "00-6d7e8f90…-02"
}
```

Client reaction: Remove the expired token, redirect to sign-in and return to "My tickets" after a successful sign-in.

429 and 500 — see §2.